

Anexo: Projeto TE

Título

Refatorização e Desenvolvimento da Aplicação Web com Framework Front-end e API de Gestão de Utilizadores para o Centro Académico Clínico dos Açores (CACA)

Contexto

Os projetos PEI1, PEI2 e PEI3 focaram no desenvolvimento faseado da presença web do Centro Académico Clínico dos Açores (CACA), uma iniciativa entre instituições de ensino superior, de saúde e de investigação nos Açores. Inicialmente, foi criada uma *landing page* estática (PEI1), seguida da adição de interatividade e animações com JavaScript (PEI2) e, posteriormente, da integração de funcionalidades de CRUD com IndexedDB e do consumo de APIs externas (PEI3).

Com a evolução das necessidades do CACA e a complexidade crescente do código, tornam-se imperativas a refatorização e unificação destas funcionalidades numa aplicação web moderna e escalável. Este projeto visa consolidar os desenvolvimentos anteriores, aproveitando uma *framework* front-end JavaScript e implementando uma API robusta para gestão de utilizadores, garantindo uma experiência de utilizador otimizada e uma arquitetura de software escalável.

Objetivos

1. Refatorizar as funcionalidades existentes (em PEI1, PEI2, PEI3) para uma *framework* front-end JavaScript (e.g., React, Vue, Angular).
2. Desenvolver uma API de gestão de utilizadores (com funcionalidades de registo, autenticação e perfis) que interaja com o *front-end*.
3. Assegurar a manutenção da identidade visual e experiência de utilizador estabelecidas nos projetos anteriores.
4. Implementar uma arquitetura modular e escalável que permita a expansão e a manutenção futuras com facilidade.
5. Garantir a responsividade da aplicação em diferentes dispositivos e acessibilidade para diversos utilizadores.
6. Aplicar boas práticas de desenvolvimento web, incluindo a organização do código, a segurança e o desempenho.

Tarefas

Cada equipa de alunos é responsável por conceber, desenvolver e refatorizar a aplicação web do CACA, aplicando os conhecimentos adquiridos e explorando novas tecnologias:

1. Análise e Design da Arquitetura:

- Definir a estrutura da aplicação utilizando a *framework* front-end escolhida.
- Projetar a arquitetura da API de gestão de utilizadores (endpoints, modelos de dados, autenticação).
- Rever e adaptar o design da *landing page* e funcionalidades existentes para a nova arquitetura.

2. Desenvolvimento Front-end:

- Implementar as funcionalidades da *landing page* (PEI1) na *framework* escolhida.
- Integrar as interações e animações (PEI2) de forma eficiente.
- Migrar as funcionalidades de CRUD com IndexedDB (PEI3) para a nova estrutura, utilizando o armazenamento local ou integrando com a API (se aplicável para dados não sensíveis).
- Desenvolver novas interfaces para a gestão de utilizadores (registo, login, perfil).
- Garantir a comunicação assíncrona com a API de *backend*.

3. Desenvolvimento Back-end (API de Gestão de Utilizadores):

- Considerar a tecnologia Node.js com Express para a construção da API.
- Implementar *endpoints* para:
 - Registo de novos utilizadores.
 - Autenticação (login) de utilizadores (e.g., JWT).
 - Gestão de perfis de utilizadores (edição, visualização).
 - Implementação de um sistema de permissões básicas (e.g., administrador, utilizador regular).
- Utilizar uma base de dados adequada para o armazenamento de dados de utilizadores (e.g., MongoDB, PostgreSQL, MySQL).

4. Integração e Testes:

- Integrar o *front-end* com a API de *backend*.
- Realizar testes de unidade e integração para ambas as partes da aplicação.
- Assegurar a correção funcional, a responsividade e a acessibilidade.

5. Documentação:

- Elaborar um README.md detalhado.
- Documentar as decisões de *design* e implementação.

Estrutura do Código e Boas Práticas

- O código JavaScript (front-end) deve ser bem organizado, modularizado em componentes/módulos e seguir as convenções da *framework* escolhida.
- O código da API (back-end) deve ser estruturado de forma lógica, com separação de responsabilidades (controladores, modelos, rotas).
- Utilizar variáveis e funções de forma clara e legível, com comentários em docstring quando necessário.
- Garantir a acessibilidade da aplicação (navegação por teclado, descrições alternativas para elementos dinâmicos, semântica HTML).
- Implementar mecanismos de segurança na API (e.g., validação de entrada, *hashing* de senhas, proteção contra CSRF/XSS).
- Otimizar o desempenho da aplicação, tanto no *front-end* (e.g., lazy loading, otimização de imagens) quanto no *back-end* (e.g., consultas eficientes à base de dados).

Entregas Obrigatórias

1. **Repositório Git (e.g., GitHub):**
 - O repositório deve conter todo o código-fonte da aplicação (front-end e back-end).
 - Histórico de *commits* claro e descritivo, que reflete o trabalho da equipa.
2. **Ficheiro README.md:**
 - **Identificação do grupo:** Nomes e números de aluno.
 - **Breve descrição da *framework* front-end e tecnologias back-end escolhidas:** Justificação das escolhas.
 - **Estrutura da Aplicação:** Diagrama ou descrição da arquitetura (front-end, back-end, base de dados, comunicação entre eles).
 - **Funcionalidades Implementadas:** Detalhes das funcionalidades migradas e das novas funcionalidades da API de gestão de utilizadores.
 - **Como Correr a Aplicação:** Instruções claras para configurar e executar tanto o *front-end* quanto o *back-end*.
 - **Quaisquer decisões de design ou implementação relevantes:** Desafios, dificuldades encontradas e soluções adotadas.
 - **Principais preocupações de acessibilidade, responsividade e segurança** adotadas.
 - **Descrição de APIs externas utilizadas** (se houver) e como foram integradas.

Organização do Trabalho e Prazos

- O projeto será realizado em equipas de 4 estudantes, com inscrição por meio de uma sondagem na página da unidade curricular no Moodle.
- Todos os elementos do grupo devem participar em todas as etapas do desenvolvimento do projeto.
- Recomenda-se que todas as tarefas sejam conceptualizadas, organizadas e executadas por todos os membros da equipa, com reuniões regulares de coordenação e de monitorização do desenvolvimento do projeto.

Prazos:

- **Data de publicação do enunciado do projeto:** ?? de maio de 2026.
- **Data-limite para a entrega do projeto:** 28 de maio de 2026.
 - Entrega do endereço do GitHub (onde está a ser desenvolvido o projeto) através da página da unidade curricular no Moodle na secção 'Projeto em Equipa 4 (PEI4)'.
- **Data da apresentação, discussão e avaliação do projeto:** 28 de maio de 2026.

CrITÉrios de Avaliação

1. **Funcionalidade e Correção (35%):**
 - Implementação robusta e correta das funcionalidades existentes refatorizadas.
 - Implementação completa e funcional da API de gestão de utilizadores (registo, login, perfil, permissões).
 - Integração eficaz entre front-end e back-end.
2. **Qualidade do Código (25%):**
 - **Front-end (15%):** Organização, legibilidade, modularidade, eficiência e uso de boas práticas da *framework* JavaScript escolhida.
 - **Back-end (10%):** Estrutura da API, legibilidade, organização e eficiência.
3. **Arquitetura e Design (20%):**
 - Adequação da *framework* front-end e tecnologias back-end escolhidas.
 - Modularidade e escalabilidade da solução.
 - Manutenção da identidade visual e princípios de UI/UX.
 - Design responsivo e acessibilidade.
4. **Segurança (10%):**
 - Implementação de mecanismos de segurança na API (autenticação, validação).
 - Proteção contra vulnerabilidades comuns.
5. **Documentação e Apresentação (10%):**
 - Clareza e detalhe do ficheiro README.md.

- Compreensão e comunicação das decisões de *design* e implementação durante a apresentação.

Observações Finais

- Podem ser usados conteúdos textuais e imagens fictícias, desde que adequados ao contexto e respeitem os direitos de autor.
- O docente poderá solicitar uma breve apresentação do projeto em aula, para discussão das opções de *design* e implementação.